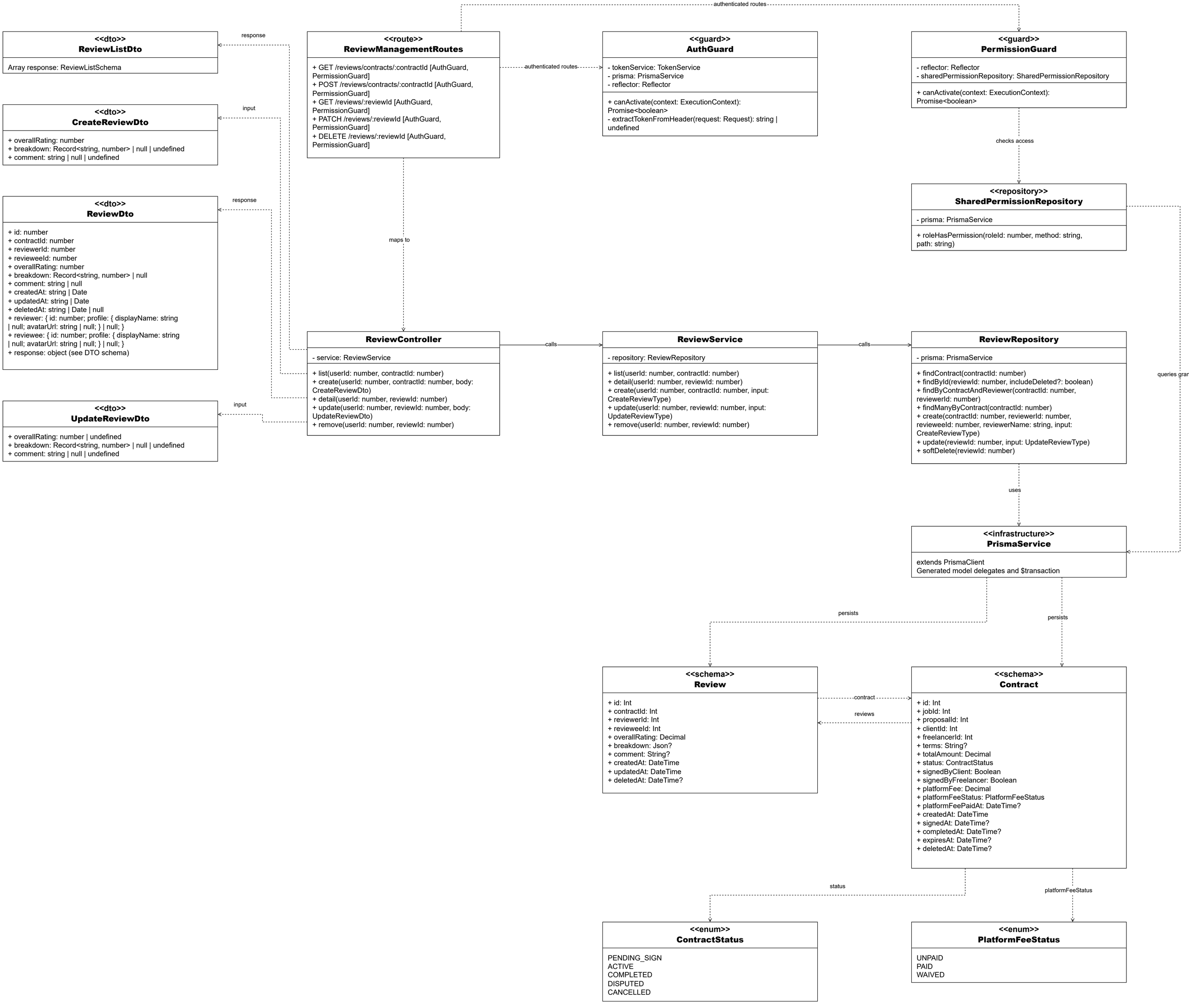

<<dto>>
ReviewListDto
Array response: ReviewListSchema
response
<<route>>
ReviewManagementRoutes
+ GET /reviews/contracts/:contractId [AuthGuard, PermissionGuard]
+ POST /reviews/contracts/:contractId [AuthGuard, PermissionGuard]
+ GET /reviews/:reviewId [AuthGuard, PermissionGuard]
+ PATCH /reviews/:reviewId [AuthGuard, PermissionGuard]
+ DELETE /reviews/:reviewId [AuthGuard, PermissionGuard]
authenticated routes
<<guard>>
AuthGuard
- tokenService: TokenService
- prisma: PrismaService
- reflector: Reflector
+ canActivate(context: ExecutionContext): Promise<boolean>
- extractTokenFromHeader(request: Request): string | undefined
authenticated routes
<<guard>>
PermissionGuard
- reflector: Reflector
- sharedPermissionRepository: SharedPermissionRepository
+ canActivate(context: ExecutionContext): Promise<boolean>
checks access
<<dto>>
CreateReviewDto
+ overallRating: number
+ breakdown: Record<string, number> | null | undefined
+ comment: string | null | undefined
input
<<repository>>
SharedPermissionRepository
- prisma: PrismaService
+ roleHasPermission(roleId: number, method: string, path: string)
<<dto>>
ReviewDto
+ id: number
+ contractId: number
+ reviewerId: number
+ revieweeId: number
+ overallRating: number
+ breakdown: Record<string, number> | null
+ comment: string | null
+ createdAt: string | Date
+ updatedAt: string | Date
+ deletedAt: string | Date | null
+ reviewer: { id: number; profile: { displayName: string | null; avatarUrl: string | null; } | null; }
+ reviewee: { id: number; profile: { displayName: string | null; avatarUrl: string | null; } | null; }
+ response: object (see DTO schema)
response
maps to
ReviewController
- service: ReviewService
+ list(userId: number, contractId: number)
+ create(userId: number, contractId: number, body: CreateReviewDto)
+ detail(userId: number, reviewId: number)
+ update(userId: number, reviewId: number, body: UpdateReviewDto)
+ remove(userId: number, reviewId: number)
calls
ReviewService
- repository: ReviewRepository
+ list(userId: number, contractId: number)
+ detail(userId: number, reviewId: number)
+ create(userId: number, contractId: number, input: CreateReviewType)
+ update(userId: number, reviewId: number, input: UpdateReviewType)
+ remove(userId: number, reviewId: number)
calls
ReviewRepository
- prisma: PrismaService
+ findContract(contractId: number)
+ findById(reviewId: number, includeDeleted?: boolean)
+ findByContractAndReviewer(contractId: number, reviewerId: number)
+ findManyByContract(contractId: number)
+ create(contractId: number, reviewerId: number, revieweeId: number, reviewerName: string, input: CreateReviewType)
+ update(reviewId: number, input: UpdateReviewType)
+ softDelete(reviewId: number)
queries grants
<<dto>>
UpdateReviewDto
+ overallRating: number | undefined
+ breakdown: Record<string, number> | null | undefined
+ comment: string | null | undefined
input
uses
<<infrastructure>>
PrismaService
extends PrismaClient
Generated model delegates and $transaction
persists
persists
<<schema>>
Review
+ id: Int
+ contractId: Int
+ reviewerId: Int
+ revieweeId: Int
+ overallRating: Decimal
+ breakdown: Json?
+ comment: String?
+ createdAt: DateTime
+ updatedAt: DateTime
+ deletedAt: DateTime?
contract
reviews
<<schema>>
Contract
+ id: Int
+ jobId: Int
+ proposalId: Int
+ clientId: Int
+ freelancerId: Int
+ terms: String?
+ totalAmount: Decimal
+ status: ContractStatus
+ signedByClient: Boolean
+ signedByFreelancer: Boolean
+ platformFee: Decimal
+ platformFeeStatus: PlatformFeeStatus
+ platformFeePaidAt: DateTime?
+ createdAt: DateTime
+ signedAt: DateTime?
+ completedAt: DateTime?
+ expiresAt: DateTime?
+ deletedAt: DateTime?
status
platformFeeStatus
<<enum>>
ContractStatus
PENDING_SIGN
ACTIVE
COMPLETED
DISPUTED
CANCELLED
<<enum>>
PlatformFeeStatus
UNPAID
PAID
WAIVED